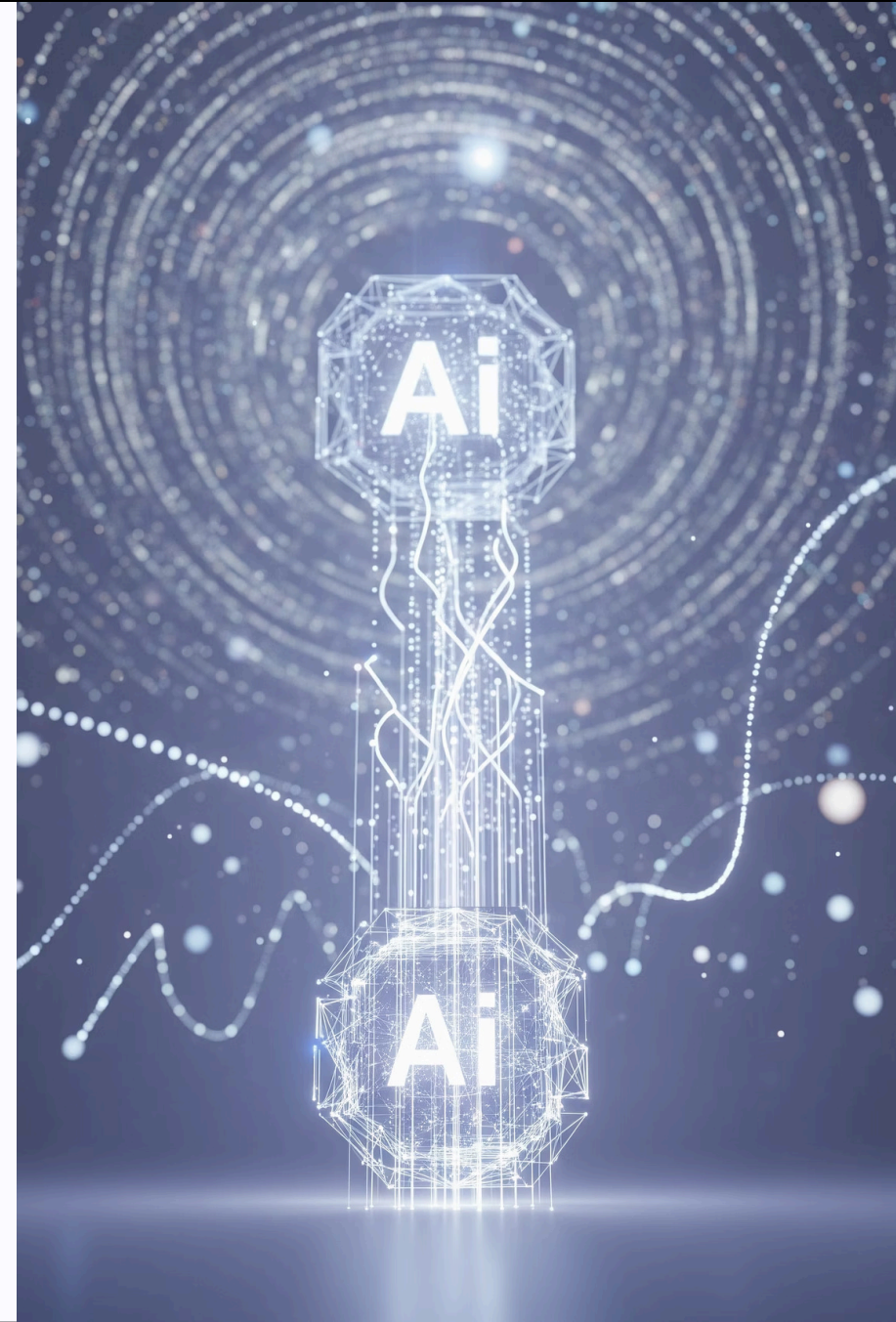


Streaming Data with Kafka & Pub/Sub

A practical guide to building real-time AI pipelines using streaming technologies

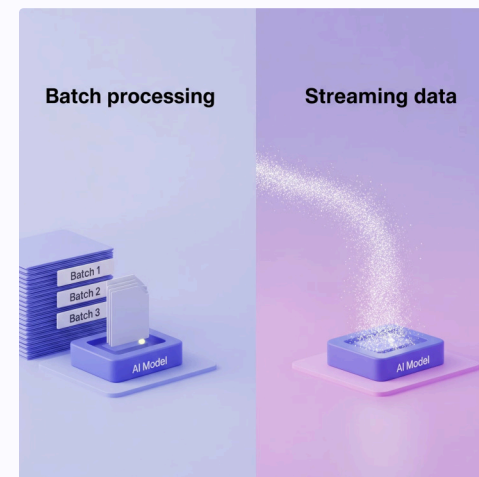


Why Streaming Data for AI?

Modern AI applications increasingly demand **real-time data processing** capabilities across various domains:

- Fraud detection systems requiring instant analysis
- Recommendation engines adapting to user behavior
- IoT networks generating continuous sensor data

Traditional batch processing creates **unacceptable latency** for time-sensitive AI applications.



Streaming infrastructure enables:

- Continuous ingestion and processing
- Low-latency ML inference
- Essential foundation for production AI systems

What Is Apache Kafka?



Distributed Platform

Open-source streaming platform designed to handle millions of events per second with fault tolerance and horizontal scalability



Topic-Based Architecture

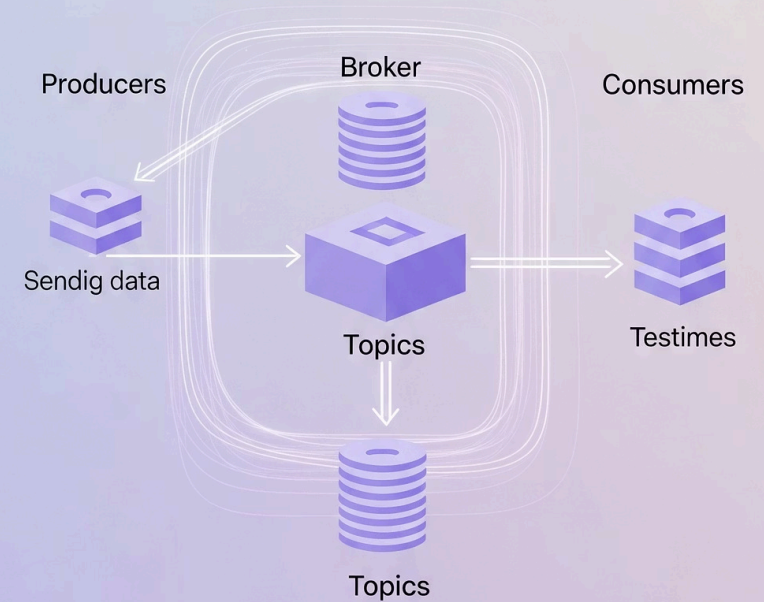
Uses [topics](#) to organize data streams where producers write data and consumers read from these channels



Rich Ecosystem

Powerful tooling including Kafka Streams for processing, ksqlDB for SQL-like operations, and Connectors for integration with external systems

Apache Kafka



Kafka in AI Workflows

IoT & Sensor Data

Stream continuous sensor readings directly into AI models for real-time analysis and predictive maintenance

User Interactions

Capture clicks, views, and interactions to power dynamic recommendation systems that adapt instantly

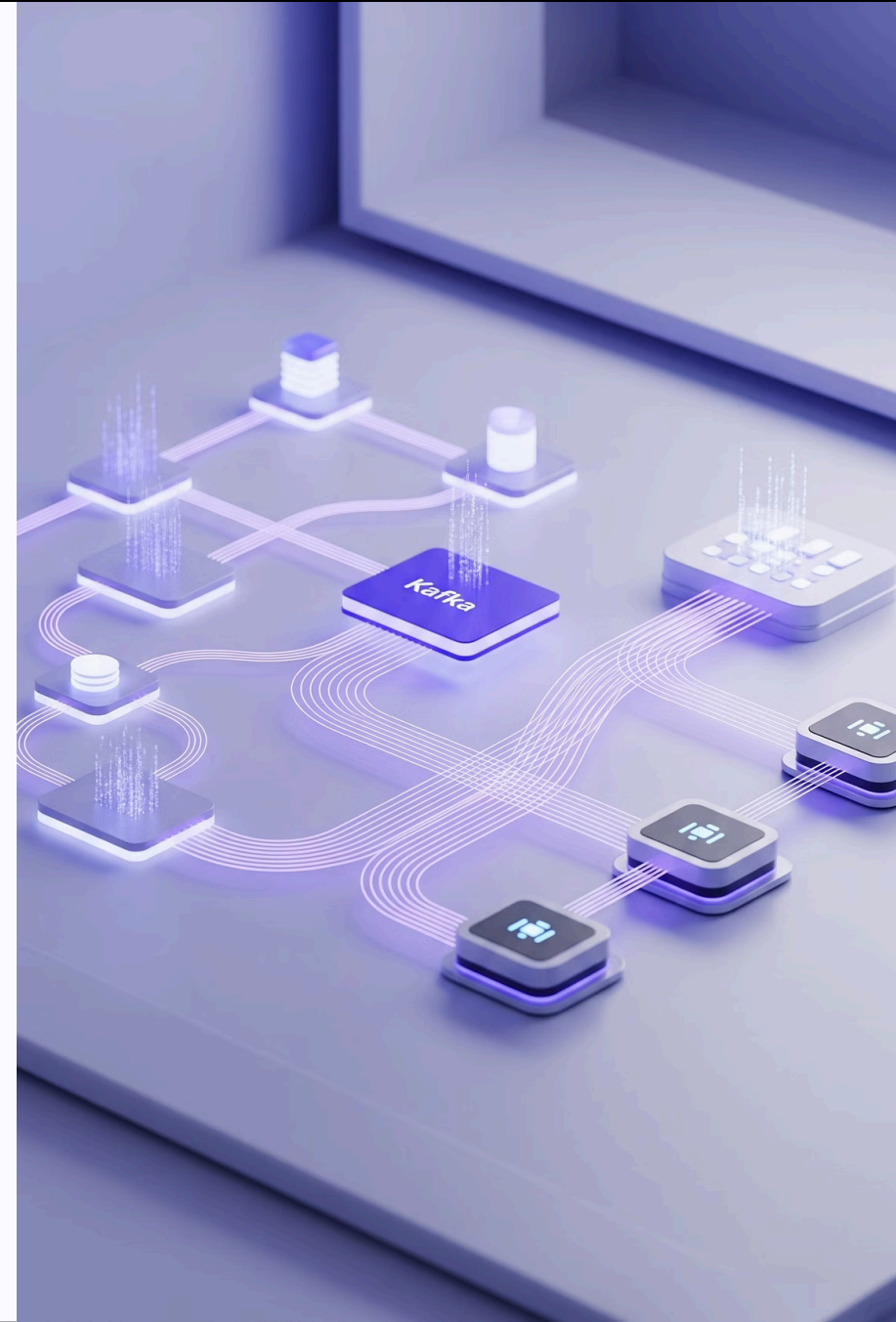
Log Analytics

Aggregate application and system logs for anomaly detection and security monitoring

Feature Pipelines

Continuously update feature stores with fresh data to improve model accuracy and relevance

Kafka's partitioning model enables parallel processing across [thousands of producers and consumers](#), making it ideal for high-scale AI workloads.



What Is Google Pub/Sub?

Google Cloud's fully managed [messaging and streaming service](#) designed for seamless integration with other cloud services:

- No infrastructure management required
- Flexible delivery modes (push or pull)
- Native integration with BigQuery, Dataflow, and Vertex AI
- Global scale with pay-per-use pricing model
- Built-in security and compliance controls

Ideal for organizations already invested in the Google Cloud ecosystem or those wanting minimal operational overhead.



- ❶ Pub/Sub automatically scales to handle traffic spikes without manual provisioning, making it particularly well-suited for variable AI workloads.

Kafka vs Pub/Sub: Choosing Your Streaming Platform



Apache Kafka

- Open-source, community-driven development
- Self-managed or Confluent Cloud options
- Greater flexibility and control
- Richer ecosystem of tools and connectors
- Strong for high-throughput use cases

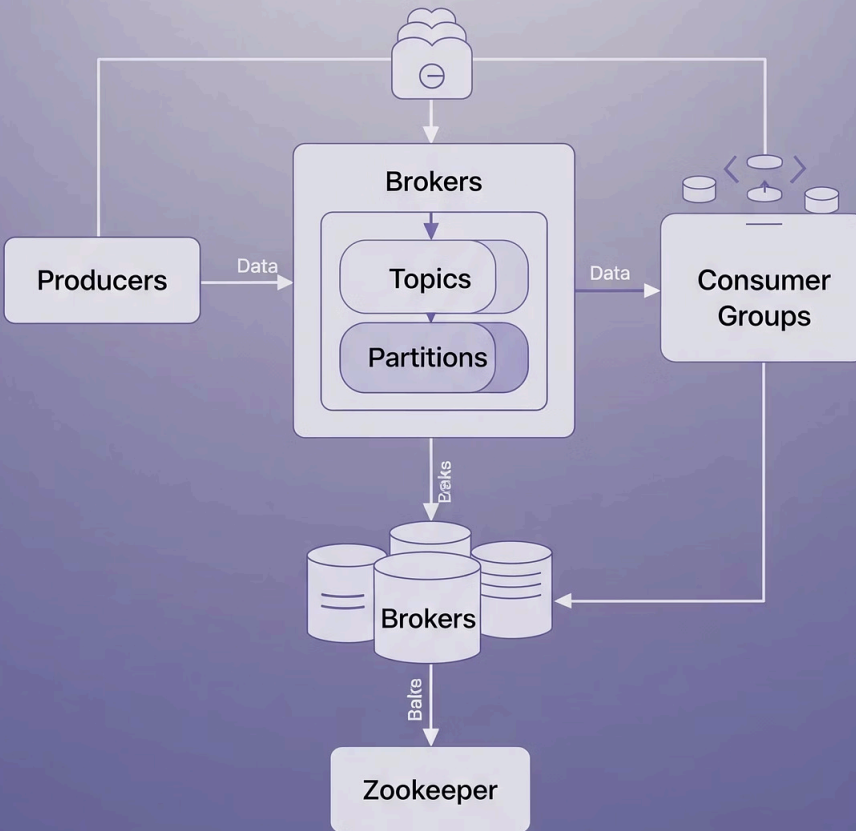


Google Pub/Sub

- Fully managed GCP service
- Zero infrastructure management
- Simpler operations and setup
- Seamless Google Cloud integration
- Pay-per-use pricing model

Both platforms provide robust foundations for [real-time AI pipelines](#), with the choice depending on your specific requirements for control, integration, and operational overhead.

Apache Kafka



Kafka Architecture



Producers

Applications that send data to Kafka topics



Brokers

Servers that store data in partitioned topics



Consumers

Applications that subscribe to and process events

Zookeeper (Legacy)

Manages cluster coordination and metadata

Kafka Connect

Integrates with databases, cloud services, and ML pipelines

Pub/Sub Architecture

Publisher

Application that creates and sends messages to a topic

Topic

Named resource that represents a feed of messages

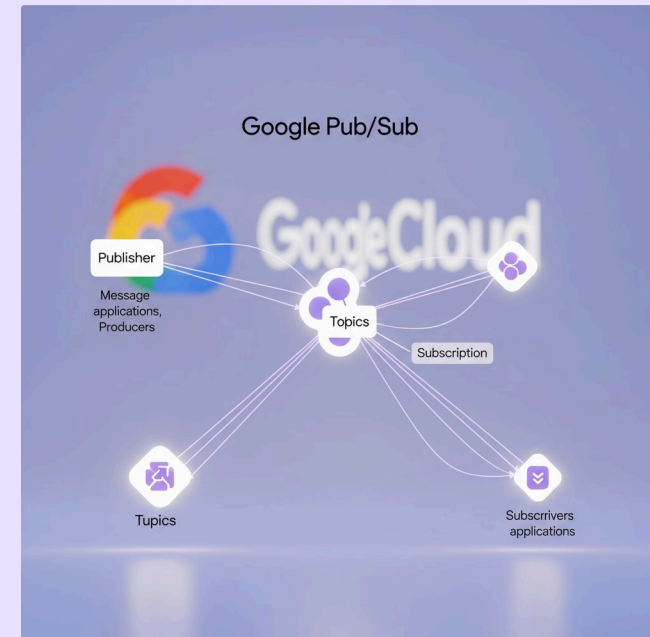
Subscription

Named resource representing the stream of messages from a topic to a specific consumer

Subscriber

Application that receives and processes messages from a subscription

Google manages all underlying infrastructure, including [automatic scaling](#) and sophisticated retry logic for failed deliveries.



Pub/Sub maintains a separation between publishing and subscribing, allowing independent scaling of each component.

AI Use Cases of Streaming

Fraud Detection

Stream financial transactions in real-time to ML models for immediate identification of suspicious patterns and prevention of fraudulent activities

Predictive Maintenance

Process continuous IoT sensor data from industrial equipment to detect anomalies and predict failures before they occur

Recommender Systems

Analyze user clickstreams and behavior patterns to generate personalized recommendations that adapt in real-time

Conversational AI

Stream messages to NLP models for real-time understanding and generation of contextually appropriate responses

Streaming platforms also enable real-time [data enrichment](#) by joining live data with feature stores for enhanced model performance.



Best Practices for AI Streaming

1 Optimize Partitioning

Design partition strategies to enable parallel AI processing and maximize throughput while maintaining message ordering when needed

2 Monitor Consumer Lag

Track and alert on processing delays to ensure AI models receive data within acceptable latency thresholds

3 Implement Security Controls

Secure streams with IAM policies, encryption, and access controls to protect sensitive data used in AI applications

4 Use Schema Management

Implement schema registry for data validation and evolution to ensure AI feature consistency



- ❏ Pair streaming platforms with specialized ML serving infrastructure like NVIDIA Triton or TensorFlow Serving for optimal performance.

Key Takeaways

Essential Knowledge

Streaming data platforms have become fundamental infrastructure for real-time AI applications, enabling continuous data flow and low-latency inference.



Kafka

Open-source, flexible, powerful with rich ecosystem



Pub/Sub

Managed, simple, cloud-native with seamless GCP integration



Next Steps

- Assess your AI workload requirements and infrastructure preferences
- Explore integration options with feature stores and ML pipelines
- Test performance characteristics with your specific data volumes
- Develop monitoring strategies for production deployment